Namespace Func.Redis

Classes

ConnectionMultiplexerProvider

RedisSourcesProvider

Structs

Error

Interfaces

IConnectionMultiplexerProvider

IRedisService

ISourcesProvider

Class ConnectionMultiplexerProvider

Namespace: Func.Redis

Assembly: Func.Redis.dll

```
public class ConnectionMultiplexerProvider : IConnectionMultiplexerProvider
```

Inheritance

object ← ConnectionMultiplexerProvider

Implements

IConnectionMultiplexerProvider

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

ConnectionMultiplexerProvider(RedisConfiguration)

```
public ConnectionMultiplexerProvider(RedisConfiguration config)
```

Parameters

`config` RedisConfiguration

Methods

GetMultiplexer()

Get the connection multiplexer

```
public IConnectionMultiplexer GetMultiplexer()
```

Returns

IConnectionMultiplexer

Struct Error

Namespace: Func.Redis

Assembly: Func.Redis.dll

```
public readonly struct Error
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Properties

Message

```
public string Message { get; }
```

Property Value

string

Methods

New(Exception)

```
[Pure]
public static Error New(Exception ex)
```

Parameters

ex Exception

Returns

Error

New(string)

```
[Pure]
public static Error New(string message)
```

Parameters

`message` string

Returns

Error

Interface IConnectionMultiplexerProvider

Namespace: Func.Redis

Assembly: Func.Redis.dll

```
public interface IConnectionMultiplexerProvider
```

Methods

GetMultiplexer()

Get the connection multiplexer

```
IConnectionMultiplexer GetMultiplexer()
```

Returns

IConnectionMultiplexer

Interface IRedisService

Namespace: Func.Redis

Assembly: Func.Redis.dll

```
public interface IRedisService
```

Methods

ExecuteAsync<T>(Func<IDatabase, Task<T>>)

Execute a Redis command

```
Task<Either<Error, T>> ExecuteAsync<T>(Func<IDatabase, Task<T>> exec)
```

Parameters

`exec` Func<IDatabase, Task<T>>

Returns

Task<Either<Error, T>>

Type Parameters

`T`

ExecuteAsync<TIn, TOut>(Func<IDatabase, Task<TIn>>, Func<TIn, TOut>)

Execute a Redis command and map the result

```
Task<Either<Error, TOut>> ExecuteAsync<TIn, TOut>(Func<IDatabase, Task<TIn>> exec,
Func<TIn, TOut> map)
```

Parameters

`exec` Func<IDatabase, Task<TIn>>

`map` Func<TIn, TOut>

Returns

Task<Either<Error, TOut>>

Type Parameters

`TIn`

`TOut`

ExecuteUnsafeAsync<TIn, TOut>(Func<IDatabase, Task<TIn>>, Func<TIn, TOut>)

Execute a Redis command and map the result. This method explicitely catches exceptions of the map Func

```
Task<Either<Error, TOut>> ExecuteUnsafeAsync<TIn, TOut>(Func<IDatabase, Task<TIn>>
exec, Func<TIn, TOut> map)
```

Parameters

`exec` Func<IDatabase, Task<TIn>>

`map` Func<TIn, TOut>

Returns

Task<Either<Error, TOut>>

Type Parameters

`TIn`

`TOut`

Execute<T>(Func<IDatabase, T>)

Execute a Redis command

```
Either<Error, T> Execute<T>(Func<IDatabase, T> exec)
```

Parameters

`exec` Func<IDatabase, T>

Returns

Either<Error, T>

Type Parameters

`T`

Execute<TIn, TOut>(Func<IDatabase, TIn>, Func<TIn, TOut>)

Execute a Redis command and map the result

```
Either<Error, TOut> Execute<TIn, TOut>(Func<IDatabase, TIn> exec, Func<TIn,
TOut> map)
```

Parameters

`exec` Func<IDatabase, TIn>

`map` Func<TIn, TOut>

Returns

Either<Error, TOut>

Type Parameters

`TIn`

TOut

Interface ISourcesProvider

Namespace: Func.Redis

Assembly: Func.Redis.dll

```
public interface ISourcesProvider
```

Methods

GetDatabase()

Get the database instance of the Redis server

```
IDatabase GetDatabase()
```

Returns

IDatabase

GetServers()

Get all the server references of the Redis instance

```
IServer[] GetServers()
```

Returns

IServer[]

Class RedisSourcesProvider

Namespace: Func.Redis

Assembly: Func.Redis.dll

```
public class RedisSourcesProvider : ISourcesProvider
```

Inheritance

object ← RedisSourcesProvider

Implements

ISourcesProvider

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

RedisSourcesProvider(IConnectionMultiplexerProvider)

```
public RedisSourcesProvider(IConnectionMultiplexerProvider provider)
```

Parameters

`provider` IConnectionMultiplexerProvider

Methods

GetDatabase()

Get the database instance of the Redis server

```
public IDatabase GetDatabase()
```

Returns

IDatabase

GetServers()

Get all the server references of the Redis instance

```
public IServer[] GetServers()
```

Returns

IServer[]

Namespace Func.Redis.Extensions

Classes

EitherLoggingExtensions

ServiceCollectionExtensions

WebApplicationExtensions

Enums

RedisCapabilities

Class EitherLoggingExtensions

Namespace: Func.Redis.Extensions

Assembly: Func.Redis.dll

```
public static class EitherLoggingExtensions
```

Inheritance

object ← EitherLoggingExtensions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Enum RedisCapabilities

Namespace: Func.Redis.Extensions

Assembly: Func.Redis.Extensions.dll

```
[Flags]
public enum RedisCapabilities
```

Fields

`Generic = 1`

`HashSet = 4`

`Key = 2`

`List = 16`

`Publish = 32`

`Set = 8`

`Subscribe = 64`

Class ServiceCollectionExtensions

Namespace: Func.Redis.Extensions

Assembly: Func.Redis.Extensions.dll

```
public static class ServiceCollectionExtensions
```

Inheritance

object ← ServiceCollectionExtensions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

AddRedisSerDes<T>(IServiceCollection)

Add a custom implementation of IRedisSerDes

```
public static IServiceCollection AddRedisSerDes<T>(this IServiceCollection services)
where T : IRedisSerDes
```

Parameters

`services` IServiceCollection

Returns

IServiceCollection

Type Parameters

`T`

AddRedis<T>(IServiceCollection, IConfiguration, RedisCapabilities, bool, params Assembly[])

Add Redis services to the service collection

```
public static IServiceCollection AddRedis<T>(this IServiceCollection services,
IConfiguration config, RedisCapabilities capabilities, bool addLogging = false,
params Assembly[] assemblies) where T : IRedisSerDes
```

Parameters

`services` IServiceCollection

`config` IConfiguration

Configuration provider to retrieve configuration parametes for RedisConfiguration and RedisKeyConfiguration

`capabilities` RedisCapabilities

Specify capabilities to be enabled

`addLogging` bool

Enable/disable loggind

`assemblies` Assembly[]

Specifiy assemblies to be scanned for IRedisSubscriber implementations

Returns

IServiceCollection

Type Parameters

`T`

Exceptions

KeyNotFoundException

AddSystemJsonRedisSerDes(IServiceCollection)

Add SystemJsonRedisSerDes Redis serializer/deserializer

```
public static IServiceCollection AddSystemJsonRedisSerDes(this
IServiceCollection services)
```

Parameters

`services` IServiceCollection

Returns

IServiceCollection

Class WebApplicationExtensions

Namespace: Func.Redis.Extensions

Assembly: Func.Redis.Extensions.dll

```
public static class WebApplicationExtensions
```

Inheritance

object ← WebApplicationExtensions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

UseRedisSubcribers(WebApplication)

```
public static WebApplication UseRedisSubcribers(this WebApplication app)
```

Parameters

`app` WebApplication

Returns

WebApplication

Namespace Func.Redis.HashSet

Interfaces

IRedisHashSetService

Interface IRedisHashSetService

Namespace: Func.Redis.HashSet

Assembly: Func.Redis.dll

```
public interface IRedisHashSetService
```

Methods

Delete(string, string)

Delete a field from a key

```
Either<Error, Unit> Delete(string key, string field)
```

Parameters

`key` string

`field` string

Returns

Either<Error, Unit>

Delete(string, params string[])

Delete multiple fields from a key

```
Either<Error, Unit> Delete(string key, params string[] fields)
```

Parameters

`key` string

`fields` string[]

Returns

Either<Error, Unit>

DeleteAsync(string, string)

Delete a field from a key

```
Task<Either<Error, Unit>> DeleteAsync(string key, string field)
```

Parameters

`key` string

`field` string

Returns

Task<Either<Error, Unit>>

DeleteAsync(string, params string[])

Delete multiple fields from a key

```
Task<Either<Error, Unit>> DeleteAsync(string key, params string[] fields)
```

Parameters

`key` string

`fields` string[]

Returns

Task<Either<Error, Unit>>

Get(string, params (Type, string)[])

Get the values stored at multiple fields of a key. When the key or field does not exist, return None

```
Either<Error, Option<object>[]> Get(string key, params (Type, string)[] typeFields)
```

Parameters

`key` string

`typeFields` (Type, string)[]

Returns

Either<Error, Option<object>[]>

GetAllAsync<T>(string)

Get all the fields and values stored at a key. When the key does not exist, return None

```
Task<Either<Error, Option<(string, T)[]>>> GetAllAsync<T>(string key)
```

Parameters

`key` string

Returns

Task<Either<Error, Option<(string, T)[]>>>

Type Parameters

`T`

GetAll<T>(string)

Get all the fields and values stored at a key. When the key does not exist, return None

```
Either<Error, Option<(string, T)[]>> GetAll<T>(string key)
```

Parameters

`key` string

Returns

Either<Error, Option<(string, T)[]>>

Type Parameters

`T`

GetAsync(string, params (Type, string)[])

```
Task<Either<Error, Option<object>[]>> GetAsync(string key, params (Type, string)
[] typeFields)
```

Parameters

`key` string

`typeFields` (Type, string)[]

Returns

Task<Either<Error, Option<object>[]>>

GetAsync<T>(string, string)

Get the value stored at a field of a key. When the key or field does not exist, return None

```
Task<Either<Error, Option<T>>> GetAsync<T>(string key, string field)
```

Parameters

`key` string

`field` string

Returns

Task<Either<Error, Option<T>>>

Type Parameters

`T`

GetAsync<T>(string, params string[])

Get the values stored at multiple fields of a key. When the key or field does not exist, return None

```
Task<Either<Error, Option<T>[]>> GetAsync<T>(string key, params string[] fields)
```

Parameters

`key` string

`fields` string[]

Returns

Task<Either<Error, Option<T>[]>>

Type Parameters

`T`

GetFieldKeys(string)

Get all the fields stored at a key. When the key does not exist, return None

```
Either<Error, Option<string[]>> GetFieldKeys(string key)
```

Parameters

`key` string

Returns

Either<Error, Option<string[]>>

GetFieldKeysAsync(string)

Get all the fields stored at a key. When the key does not exist, return None

```
Task<Either<Error, Option<string[]>>> GetFieldKeysAsync(string key)
```

Parameters

`key` string

Returns

Task<Either<Error, Option<string[]>>>

GetValuesAsync<T>(string)

Get the values stored at all fields of a key. When the key does not exist, return None

```
Task<Either<Error, Option<T[]>>> GetValuesAsync<T>(string key)
```

Parameters

`key` string

Returns

Task<Either<Error, Option<T[]>>>

Type Parameters

`T`

GetValues<T>(string)

Get the values stored at all fields of a key. When the key does not exist, return None

```
Either<Error, Option<T[]>> GetValues<T>(string key)
```

Parameters

`key` string

Returns

Either<Error, Option<T[]>>

Type Parameters

`T`

Get<T>(string, string)

Get the value stored at a field of a key. When the key or field does not exist, return None

```
Either<Error, Option<T>> Get<T>(string key, string field)
```

Parameters

`key` string

`field` string

Returns

Either<Error, Option<T>>

Type Parameters

`T`

Get<T>(string, params string[])

Get the values stored at multiple fields of a key. When the key or field does not exist, return None

```
Either<Error, Option<T>[]> Get<T>(string key, params string[] fields)
```

Parameters

`key` string

`fields` string[]

Returns

Either<Error, Option<T>[]>

Type Parameters

`T`

SetAsync<T>(string, string, T)

Set a value at a field of a key

```
Task<Either<Error, Unit>> SetAsync<T>(string key, string field, T value)
```

Parameters

`key` string

`field` string

`value` T

Returns

Task<Either<Error, Unit>>

Type Parameters

`T`

SetAsync<T>(string, params (string, T)[])

Set values at multiple fields of a key

```
Task<Either<Error, Unit>> SetAsync<T>(string key, params (string, T)[] pairs)
```

Parameters

`key` string

`pairs` (string, T)[]

Returns

Task<Either<Error, Unit>>

Type Parameters

`T`

Set<T>(string, string, T)

Set a value at a field of a key

```
Either<Error, Unit> Set<T>(string key, string field, T value)
```

Parameters

`key` string

`field` string

`value` T

Returns

Either<Error, Unit>

Type Parameters

T

Set<T>(string, params (string, T)[])

Set values at multiple fields of a key

```
Either<Error, Unit> Set<T>(string key, params (string, T)[] pairs)
```

Parameters

key string

pairs (string, T)[]

Returns

Either<Error, Unit>

Type Parameters

T

Namespace Func.Redis.Key

Interfaces

IRedisKeyService

Interface IRedisKeyService

Namespace: Func.Redis.Key

Assembly: Func.Redis.dll

```
public interface IRedisKeyService
```

Methods

Delete(string)

Delete a key

```
Either<Error, Unit> Delete(string key)
```

Parameters

`key` string

Returns

Either<Error, Unit>

Delete(params string[])

Delete multiple keys

```
Either<Error, Unit> Delete(params string[] keys)
```

Parameters

`keys` string[]

Returns

Either<Error, Unit>

DeleteAsync(string)

Delete a key

```
Task<Either<Error, Unit>> DeleteAsync(string key)
```

Parameters

`key` string

Returns

Task<Either<Error, Unit>>

DeleteAsync(params string[])

Delete multiple keys

```
Task<Either<Error, Unit>> DeleteAsync(params string[] keys)
```

Parameters

`keys` string[]

Returns

Task<Either<Error, Unit>>

GetAsync<T>(string)

Get the value stored at a key. When the key does not exist, return None

```
Task<Either<Error, Option<T>>> GetAsync<T>(string key)
```

Parameters

`key` string

Returns

Task<Either<Error, Option<T>>>

Type Parameters

`T`

GetAsync<T>(params string[])

Get the values stored at multiple keys. When a key does not exist, return None

```
Task<Either<Error, Option<T>[]>> GetAsync<T>(params string[] keys)
```

Parameters

`keys` string[]

Returns

Task<Either<Error, Option<T>[]>>

Type Parameters

`T`

GetKeys(string)

Get all keys matching a pattern

```
Either<Error, string[]> GetKeys(string pattern)
```

Parameters

`pattern` string

Returns

Either<Error, string[]>

GetKeysAsync(string)

Get all keys matching a pattern

```
Task<Either<Error, string[]>> GetKeysAsync(string pattern)
```

Parameters

`pattern` string

Returns

Task<Either<Error, string[]>>

Get<T>(string)

Get the value stored at a key. When the key does not exist, return None

```
Either<Error, Option<T>> Get<T>(string key)
```

Parameters

`key` string

Returns

Either<Error, Option<T>>

Type Parameters

`T`

Get<T>(params string[])

Get the values stored at multiple keys. When a key does not exist, return None

```
Either<Error, Option<T>[]> Get<T>(params string[] keys)
```

Parameters

`keys` string[]

Returns

Either<Error, Option<T>[]>

Type Parameters

`T`

RenameKey(string, string)

Rename a key

```
Either<Error, Unit> RenameKey(string key, string newKey)
```

Parameters

`key` string

`newKey` string

Returns

Either<Error, Unit>

RenameKeyAsync(string, string)

Rename a key

```
Task<Either<Error, Unit>> RenameKeyAsync(string key, string newKey)
```

Parameters

`key` string

`newKey` string

Returns

Task<Either<Error, Unit>>

SetAsync<T>(string, T)

Set a value at a key

```
Task<Either<Error, Unit>> SetAsync<T>(string key, T value)
```

Parameters

`key` string

`value` T

Returns

Task<Either<Error, Unit>>

Type Parameters

`T`

SetAsync<T>(params (string, T)[])

```
Task<Either<Error, Unit>> SetAsync<T>(params (string, T)[] pairs)
```

Parameters

`pairs` (string, T)[]

Returns

Task <Either<Error, Unit>>

Type Parameters

`T`

Set<T>(string, T)

Set a value at a key

```
Either<Error, Unit> Set<T>(string key, T value)
```

Parameters

`key` string

`value` T

Returns

Either<Error, Unit>

Type Parameters

`T`

Set<T>(params (string, T)[])

Set values at multiple keys

```
Either<Error, Unit> Set<T>(params (string, T)[] pairs)
```

Parameters

`pairs` (string, T)[]

Returns

Either<Error, Unit>

Type Parameters

T

Namespace Func.Redis.List

Interfaces

IRedisListService

Interface IRedisListService

Namespace: Func.Redis.List

Assembly: Func.Redis.dll

```
public interface IRedisListService
```

Methods

AppendAsync<T>(string, T)

Add an item at the end of a list

```
Task<Either<Error, Unit>> AppendAsync<T>(string key, T value)
```

Parameters

`key` string

`value` T

Returns

Task<Either<Error, Unit>>

Type Parameters

`T`

AppendAsync<T>(string, params T[])

Add multiple items at the end of a list

```
Task<Either<Error, Unit>> AppendAsync<T>(string key, params T[] values)
```

Parameters

`key` string

`values` T[]

Returns

Task<Either<Error, Unit>>

Type Parameters

`T`

Append<T>(string, T)

Add an item at the end of a list

```
Either<Error, Unit> Append<T>(string key, T value)
```

Parameters

`key` string

`value` T

Returns

Either<Error, Unit>

Type Parameters

`T`

Append<T>(string, params T[])

Add multiple items at the end of a list

```
Either<Error, Unit> Append<T>(string key, params T[] values)
```

Parameters

`key` string

`values` T[]

Returns

Either<Error, Unit>

Type Parameters

`T`

GetAsync<T>(string, long)

Get the item at a specific index. When the item does not exist, return None

```
Task<Either<Error, Option<T>>> GetAsync<T>(string key, long index)
```

Parameters

`key` string

`index` long

Returns

Task<Either<Error, Option<T>>>

Type Parameters

`T`

GetAsync<T>(string, long, long)

Get the items in an index interval. When the index have no data, return none

```
Task<Either<Error, Option<T>[]>> GetAsync<T>(string key, long start, long stop)
```

Parameters

`key` string

`start` long

`stop` long

Returns

Task<Either<Error, Option<T>[]>>

Type Parameters

`T`

Get<T>(string, long)

Get the item at a specific index. When the item does not exist, return None

```
Either<Error, Option<T>> Get<T>(string key, long index)
```

Parameters

`key` string

`index` long

Returns

Either<Error, Option<T>>

Type Parameters

`T`

Get<T>(string, long, long)

Get the items in an index interval. When the index have no data, return none

```
Either<Error, Option<T>[]> Get<T>(string key, long start, long stop)
```

Parameters

`key` string

`start` long

`stop` long

Returns

Either<Error, Option<T>[]>

Type Parameters

`T`

PopAsync<T>(string)

Removes and returns an item from the end of a list

```
Task<Either<Error, Option<T>>> PopAsync<T>(string key)
```

Parameters

`key` string

Returns

Task<Either<Error, Option<T>>>

Type Parameters

`T`

PopAsync<T>(string, long)

Removes multiple items from the end of a list

```
Task<Either<Error, Option<T>[]>> PopAsync<T>(string key, long count)
```

Parameters

`key` string

`count` long

Returns

Task<Either<Error, Option<T>[]>>

Type Parameters

`T`

Pop<T>(string)

Removes and returns an item from the end of a list

```
Either<Error, Option<T>> Pop<T>(string key)
```

Parameters

`key` string

Returns

Either<Error, Option<T>>

Type Parameters

`T`

Pop<T>(string, long)

Removes multiple items from the end of a list

```
Either<Error, Option<T>[]> Pop<T>(string key, long count)
```

Parameters

`key` string

`count` long

Returns

Either<Error, Option<T>[]>

Type Parameters

`T`

PrependAsync<T>(string, T)

Add an item at the beginning of a list

```
Task<Either<Error, Unit>> PrependAsync<T>(string key, T value)
```

Parameters

`key` string

`value` T

Returns

Task<Either<Error, Unit>>

Type Parameters

`T`

PrependAsync<T>(string, params T[])

Add multiple items at the beginning of a list

```
Task<Either<Error, Unit>> PrependAsync<T>(string key, params T[] values)
```

Parameters

`key` string

`values` T[]

Returns

Task<Either<Error, Unit>>

Type Parameters

`T`

Prepend<T>(string, T)

Add multiple items at the beginning of a list

```
Either<Error, Unit> Prepend<T>(string key, T value)
```

Parameters

`key` string

`value` T

Returns

Either<Error, Unit>

Type Parameters

`T`

Prepend<T>(string, params T[])

Add an item at the beginning of a list

```
Either<Error, Unit> Prepend<T>(string key, params T[] values)
```

Parameters

`key` string

`values` T[]

Returns

Either<Error, Unit>

Type Parameters

`T`

ShiftAsync<T>(string)

Removes an item from the beginning of a list

```
Task<Either<Error, Option<T>>> ShiftAsync<T>(string key)
```

Parameters

`key` string

Returns

Task<Either<Error, Option<T>>>

Type Parameters

`T`

ShiftAsync<T>(string, long)

Removes multiple items from the beginning of a list

```
Task<Either<Error, Option<T>[]>> ShiftAsync<T>(string key, long count)
```

Parameters

`key` string

`count` long

Returns

Task<Either<Error, Option<T>[]>>

Type Parameters

`T`

Shift<T>(string)

Removes an item from the beginning of a list

```
Either<Error, Option<T>> Shift<T>(string key)
```

Parameters

`key` string

Returns

Either<Error, Option<T>>

Type Parameters

`T`

Shift<T>(string, long)

Removes multiple items from the beginning of a list

```
Either<Error, Option<T>[]> Shift<T>(string key, long count)
```

Parameters

`key` string

`count` long

Returns

Either<Error, Option<T>[]>

Type Parameters

`T`

Size(string)

Get the size of a list

```
Either<Error, long> Size(string key)
```

Parameters

`key` string

Returns

Either<Error, long>

SizeAsync(string)

Get the size of a list

```
Task<Either<Error, long>> SizeAsync(string key)
```

Parameters

key string

Returns

Task<Either<Error, long>>

Namespace Func.Redis.Models

Classes

RedisConfiguration

RedisKeyConfiguration

Class RedisConfiguration

Namespace: Func.Redis.Models

Assembly: Func.Redis.dll

```
public record RedisConfiguration : IEquatable<RedisConfiguration>
```

Inheritance

object ← RedisConfiguration

Implements

IEquatable<RedisConfiguration>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

ConnectionString

The connection string to the Redis server

```
public string ConnectionString { get; init; }
```

Property Value

string

Class RedisKeyConfiguration

Namespace: Func.Redis.Models

Assembly: Func.Redis.dll

```
public record RedisKeyConfiguration : IEquatable<RedisKeyConfiguration>
```

Inheritance

object ← RedisKeyConfiguration

Implements

IEquatable<RedisKeyConfiguration>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

KeyPrefix

The prefix to be added to the key

```
public string KeyPrefix { get; init; }
```

Property Value

string

Namespace Func.Redis.Publisher

Interfaces

IRedisPublisherService

Interface IRedisPublisherService

Namespace: Func.Redis.Publisher

Assembly: Func.Redis.dll

```
public interface IRedisPublisherService
```

Methods

Publish(string, object)

Publish a message to a channel

```
Either<Error, Unit> Publish(string channel, object message)
```

Parameters

`channel` string

`message` object

Returns

Either<Error, Unit>

PublishAsync(string, object)

Publish a message to a channel

```
Task<Either<Error, Unit>> PublishAsync(string channel, object message)
```

Parameters

`channel` string

`message` object

Returns

Task <Either<Error, Unit>>

Namespace Func.Redis.SerDes

Interfaces

IRedisSerDes

Interface IRedisSerDes

Namespace: Func.Redis.SerDes

Assembly: Func.Redis.dll

```
public interface IRedisSerDes
```

Methods

Deserialize(RedisValue, Type)

```
Option<object> Deserialize(RedisValue value, Type type)
```

Parameters

`value` RedisValue

`type` Type

Returns

Option<object>

Deserialize<T>(HashEntry[])

```
Option<(string, T)[]> Deserialize<T>(HashEntry[] entries)
```

Parameters

`entries` HashEntry[]

Returns

Option<(string, T)[]>

Type Parameters

`T`

Deserialize<T>(RedisValue)

```
Option<T> Deserialize<T>(RedisValue value)
```

Parameters

`value` RedisValue

Returns

Option<T>

Type Parameters

`T`

Deserialize<T>(RedisValue[])

```
Option<T[]> Deserialize<T>(RedisValue[] values)
```

Parameters

`values` RedisValue[]

Returns

Option<T[]>

Type Parameters

`T`

Serialize<T>(T)

```
RedisValue Serialize<T>(T value)
```

Parameters

`value` T

Returns

RedisValue

Type Parameters

`T`

Namespace Func.Redis.SerDes.Json

Classes

SpanJsonRedisSerDes

SystemJsonRedisSerDes

Class SpanJsonRedisSerDes

Namespace: Func.Redis.SerDes.Json

Assembly: Func.Redis.dll

```
public class SpanJsonRedisSerDes : IRedisSerDes
```

Inheritance

object ← SpanJsonRedisSerDes

Implements

IRedisSerDes

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

Deserialize(RedisValue, Type)

```
public Option<object> Deserialize(RedisValue value, Type type)
```

Parameters

`value` RedisValue

`type` Type

Returns

Option<object>

Deserialize<T>(HashEntry[])

```
public Option<(string, T)[]> Deserialize<T>(HashEntry[] entries)
```

Parameters

`entries` HashEntry[]

Returns

Option<(string, T)[]>

Type Parameters

`T`

Deserialize<T>(RedisValue)

```
public Option<T> Deserialize<T>(RedisValue value)
```

Parameters

`value` RedisValue

Returns

Option<T>

Type Parameters

`T`

Deserialize<T>(RedisValue[])

```
public Option<T[]> Deserialize<T>(RedisValue[] values)
```

Parameters

`values` RedisValue[]

Returns

Option<T[]>

Type Parameters

`T`

Serialize<T>(T)

```
public RedisValue Serialize<T>(T value)
```

Parameters

`value` T

Returns

RedisValue

Type Parameters

`T`

Class SystemJsonRedisSerDes

Namespace: Func.Redis.SerDes.Json

Assembly: Func.Redis.dll

```
public class SystemJsonRedisSerDes : IRedisSerDes
```

Inheritance

object ← SystemJsonRedisSerDes

Implements

IRedisSerDes

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

Deserialize(RedisValue, Type)

```
public Option<object> Deserialize(RedisValue value, Type type)
```

Parameters

`value` RedisValue

`type` Type

Returns

Option<object>

Deserialize<T>(HashEntry[])

```
public Option<(string, T)[]> Deserialize<T>(HashEntry[] entries)
```

Parameters

`entries` HashEntry[]

Returns

Option<(string, T)[]>

Type Parameters

`T`

Deserialize<T>(RedisValue)

```
public Option<T> Deserialize<T>(RedisValue value)
```

Parameters

`value` RedisValue

Returns

Option<T>

Type Parameters

`T`

Deserialize<T>(RedisValue[])

```
public Option<T[]> Deserialize<T>(RedisValue[] values)
```

Parameters

`values` RedisValue[]

Returns

Option<T[]>

Type Parameters

`T`

Serialize<T>(T)

```
public RedisValue Serialize<T>(T value)
```

Parameters

`value` T

Returns

RedisValue

Type Parameters

`T`

Namespace Func.Redis.Set

Interfaces

IRedisSetService

Interface IRedisSetService

Namespace: Func.Redis.Set

Assembly: Func.Redis.dll

```
public interface IRedisSetService
```

Methods

AddAsync<T>(string, T)

Add a value to a set key

```
Task<Either<Error, Unit>> AddAsync<T>(string key, T value)
```

Parameters

`key` string

`value` T

Returns

Task<Either<Error, Unit>>

Type Parameters

`T`

Add<T>(string, T)

Add a value to a set key

```
Either<Error, Unit> Add<T>(string key, T value)
```

Parameters

`key` string

`value` T

Returns

Either<Error, Unit>

Type Parameters

`T`

DeleteAsync<T>(string, T)

Delete a value from a set key

```
Task<Either<Error, Unit>> DeleteAsync<T>(string key, T value)
```

Parameters

`key` string

`value` T

Returns

Task<Either<Error, Unit>>

Type Parameters

`T`

DeleteAsync<T>(string, params T[])

Delete multiple values from a set key

```
Task<Either<Error, Unit>> DeleteAsync<T>(string key, params T[] values)
```

Parameters

`key` string

`values` T[]

Returns

Task<Either<Error, Unit>>

Type Parameters

`T`

Delete<T>(string, T)

Delete a value from a set key

```
Either<Error, Unit> Delete<T>(string key, T value)
```

Parameters

`key` string

`value` T

Returns

Either<Error, Unit>

Type Parameters

`T`

Delete<T>(string, params T[])

Delete multiple values from a set key

```
Either<Error, Unit> Delete<T>(string key, params T[] values)
```

Parameters

`key` string

`values` T[]

Returns

Either<Error, Unit>

Type Parameters

`T`

DifferenceAsync<T>(string, string)

Get the difference between two set keys

```
Task<Either<Error, T[]>> DifferenceAsync<T>(string key1, string key2)
```

Parameters

`key1` string

`key2` string

Returns

Task<Either<Error, T[]>>

Type Parameters

`T`

Difference<T>(string, string)

Get the difference between two set keys

```
Either<Error, T[]> Difference<T>(string key1, string key2)
```

Parameters

`key1` string

`key2` string

Returns

Either<Error, T[]>

Type Parameters

`T`

GetAllAsync<T>(string)

Get all values from a set key

```
Task<Either<Error, Option<T>[]>> GetAllAsync<T>(string key)
```

Parameters

`key` string

Returns

Task<Either<Error, Option<T>[]>>

Type Parameters

`T`

GetAll<T>(string)

Get all values from a set key

```
Either<Error, Option<T>[]> GetAll<T>(string key)
```

Parameters

`key` string

Returns

Either<Error, Option<T>[]>

Type Parameters

`T`

IntersectAsync<T>(string, string)

Get the intersection between two set keys

```
Task<Either<Error, T[]>> IntersectAsync<T>(string key1, string key2)
```

Parameters

`key1` string

`key2` string

Returns

Task<Either<Error, T[]>>

Type Parameters

`T`

Intersect<T>(string, string)

Get the intersection between two set keys

```
Either<Error, T[]> Intersect<T>(string key1, string key2)
```

Parameters

`key1` string

`key2` string

Returns

Either<Error, T[]>

Type Parameters

`T`

PopAsync<T>(string)

Removes and returns an item from a set key. When the key does not exist, return none

```
Task<Either<Error, Option<T>>> PopAsync<T>(string key)
```

Parameters

`key` string

Returns

Task<Either<Error, Option<T>>>

Type Parameters

`T`

Pop<T>(string)

Removes and returns an item from a set key. When the key does not exist, return none

```
Either<Error, Option<T>> Pop<T>(string key)
```

Parameters

`key` string

Returns

Either<Error, Option<T>>

Type Parameters

`T`

Size(string)

Get the size of a set key

```
Either<Error, long> Size(string key)
```

Parameters

`key` string

Returns

Either<Error, long>

SizeAsync(string)

Get the size of a set key

```
Task<Either<Error, long>> SizeAsync(string key)
```

Parameters

`key` string

Returns

Task<Either<Error, long>>

UnionAsync<T>(string, string)

Get the union between two set keys

```
Task<Either<Error, T[]>> UnionAsync<T>(string key1, string key2)
```

Parameters

`key1` string

`key2` string

Returns

Task<Either<Error, T[]>>

Type Parameters

`T`

Union<T>(string, string)

Get the union between two set keys

```
Either<Error, T[]> Union<T>(string key1, string key2)
```

Parameters

`key1` string

`key2` string

Returns

Either<Error, T[]>

Type Parameters

T

Namespace Func.Redis.Subscriber

Interfaces

IRedisSubscriber

Interface IRedisSubscriber

Namespace: Func.Redis.Subscriber

Assembly: Func.Redis.dll

```
public interface IRedisSubscriber
```

Methods

GetSubscriptionHandler()

Subscribe to a channel

```
(string, Action<RedisChannel, RedisValue>) GetSubscriptionHandler()
```

Returns

(string, Action<RedisChannel, RedisValue>)